

Задание 1.2. Автоматизация установки ПО

Тихая установка программ

Под «тихой установкой» (silent install) понимается запуск установщика программы без отображения диалоговых окон и без необходимости вручную нажимать «Далее», «Установить», «Принять условия лицензии» и т. п. Все параметры установки в этом случае задаются заранее через ключи командной строки. Такой способ незаменим при автоматизации: он позволяет установить сразу несколько программ одной командой или скриптом, без участия пользователя.

Пакетный менеджер winget, встроенный в современные версии Windows, поддерживает тихую установку «из коробки» с помощью ключей --silent (не показывать интерфейс установщика) и --accept-package-agreements, --accept-source-agreements (автоматически принять лицензионные соглашения пакета и источника пакетов). Именно эти ключи используются в командах установки ниже: они гарантируют, что процесс пройдёт полностью автоматически и без запроса подтверждений.

Использование тихой установки особенно удобно при развёртывании рабочего места «с нуля» например, при настройке нового компьютера сотрудника, когда одним скриптом нужно поставить весь стандартный набор программ без постоянного присутствия человека рядом с монитором.

Сравнение BAT-сценариев и PowerShell

Для автоматизации задач в Windows чаще всего используют два инструмента: пакетные файлы (.bat/.cmd), выполняемые интерпретатором cmd.exe, и сценарии PowerShell (.ps1). У них разное назначение, и выбор между ними обычно зависит от сложности задачи.

BAT-файлы работают с простым набором команд, унаследованным ещё от MS-DOS. Их синтаксис прост и понятен даже начинающему пользователю, они запускаются без дополнительных настроек безопасности и хорошо подходят для несложных линейных сценариев например, для последовательного вызова нескольких команд winget install, как в скрипте установки ниже. Из минусов: ограниченная обработка ошибок (фактически сводится к проверке кода возврата через errorlevel), неудобная работа с текстом в кодировках, отличных от системной, и отсутствие полноценных структур данных.

PowerShell более современный и мощный инструмент. Он работает не с текстовыми строками, а с объектами (например, результат команды можно сразу передать дальше по конвейеру и обратиться к его свойствам), поддерживает конструкции try/catch для аккуратной обработки ошибок, умеет работать с JSON, XML, сетевыми запросами и модулями .NET. Однако за эти возможности приходится платить более сложным синтаксисом и тем, что по умолчанию в Windows может быть ограничена политика выполнения сценариев (Execution Policy), которую иногда нужно отдельно разрешать администратору.

На практике для простой и предсказуемой задачи установить фиксированный список программ через winget вполне достаточно BAT-сценария, что и было сделано ниже. PowerShell же оправдан, когда требуется более гибкая логика: например, читать список программ из отдельного файла, проверять, установлена ли программа, прежде чем ставить её повторно, или формировать подробный структурированный отчёт об установке.

Список используемого программного обеспечения

Для автоматической установки выбраны четыре программы, часто необходимые на новом рабочем месте разработчика: клиент Git с графическим интерфейсом, браузер, архиватор и редактор кода.

№	Программа	Назначение	Официальный сайт
1	GitHub Desktop	Клиент Git с графическим интерфейсом	https://desktop.github.com/
2	Microsoft Edge	Браузер	https://www.microsoft.com/edge
3	7-Zip	Архиватор	https://www.7-zip.org/
4	Visual Studio Code	Редактор кода	https://code.visualstudio.com/

Все четыре программы устанавливаются через встроенный в Windows пакетный менеджер winget с флагами тихой установки:

```
winget install --id GitHub.GitHubDesktop --silent --accept-package-agreements --accept-source-agreements
winget install --id Microsoft.Edge --silent --accept-package-agreements --accept-source-agreements
winget install --id 7zip.7zip --silent --accept-package-agreements --accept-source-agreements
winget install --id Microsoft.VisualStudioCode --silent --accept-package-agreements --accept-source-agreements
```

Идентификаторы пакетов указаны в соответствии с официальным каталогом winget; например, ID GitHub.GitHubDesktop подтверждён официальным манифестом winget.

Инструкция по запуску скрипта

- Скопировать install.bat на локальный диск (например, C:\install\).
- Убедиться, что учётная запись локальный администратор.
- Правый клик по install.bat → «Запуск от имени администратора».
- Дождаться сообщения «Все программы установлены».
- Проверить install_log.txt: для каждой программы должна быть строка [OK].

Листинг скрипта (install.bat)

```
install.bat
@echo off
chcp 65001 >nul
setlocal enabledelayedexpansion
set LOGFILE=%~dp0install_log.txt
echo === Начало установки: %date% %time% === >> "%LOGFILE%"

:: --- Проверка winget ---
where winget >nul 2>&1
if %errorlevel% neq 0 (
    echo [ОШИБКА] winget не найден.
    echo [ОШИБКА] winget не найден >> "%LOGFILE%"
    pause
    exit /b 1
)

:: --- Установка 4 программ ---
call :install "GitHub.GitHubDesktop" "GitHub Desktop"
call :install "Microsoft.Edge" "Microsoft Edge"
call :install "7zip.7zip" "7-Zip"
call :install "Microsoft.VisualStudioCode" "Visual Studio Code"

echo === Установка завершена: %date% %time% === >> "%LOGFILE%"
echo.
echo Все программы установлены. Лог: %LOGFILE%
pause
exit /b 0

:: ----- Подпрограмма для winget -----
:install
echo.
echo --- Установка %~2 (%~1) ---
winget install --id %~1 --silent --accept-package-agreements --accept-source-
agreements >> "%LOGFILE%" 2>&1
if !errorlevel! equ 0 (
    echo [OK] %~2 установлен.
    echo [OK] %~2 установлен >> "%LOGFILE%"
) else (
    echo [ОШИБКА] %~2, код: !errorlevel!
    echo [ОШИБКА] %~2, код: !errorlevel! >> "%LOGFILE%"
)
exit /b 0
```

Комментарии к коду

- chcp 65001 включает UTF-8 для корректного вывода русских букв.
- setlocal enabledelayedexpansion включает отложенное раскрытие переменных, без него конструкция !errorlevel! внутри блока в скобках работать не будет.
- set "LOGFILE=%~dp0install_log.txt" лог создаётся рядом со скриптом.
- where winget проверка наличия пакетного менеджера; при отсутствии скрипт выходит с кодом 1.
- call :install "ID" "Имя" подпрограмма установки; первый аргумент ID пакета winget, второй отображаемое имя.

- `winget install --silent --accept-package-agreements --accept-source-agreements` тихая установка с авто-принятием лицензий.
- `if !errorlevel! equ 0` проверка кода возврата: успех означает, что программа установлена, иначе в лог пишется код ошибки.
- `exit /b 0` корректное завершение подпрограммы и скрипта.

Тестирование

Скрипт проверен на Windows 10 Pro (22H2) и Windows 11 Pro (23H2) в аудитории кафедры. Проверены следующие сценарии: запуск с правами администратора (4 программы установлены, в логе 4 записи [OK]), запуск без прав (установка прервана), отсутствие `winget` (сообщение [ОШИБКА] `winget` не найден, выход с кодом 1), повторный запуск (`winget` пропускает уже установленные пакеты). Русские символы в выводе и логе отображаются корректно благодаря `chcp 65001`.

Отдельное замечание: при первом запуске `winget` может обновлять источник пакетов, из-за чего установка первой программы занимает до 2–3 минут.

Альтернативный вариант: тот же сценарий на PowerShell

Ниже приведён упрощённый аналог bat-скрипта, переписанный на PowerShell. Логика та же: цикл по списку программ, тихая установка через `winget` и запись результата в лог-файл, но благодаря циклу `foreach` и конструкции `try/catch` код получается короче и немного нагляднее.

install.ps1

```
$logFile = Join-Path $PSScriptRoot "install_log.txt"
"=== Начало установки: $(Get-Date) ===" | Out-File $logFile -Append

# Список программ: ID пакета winget и отображаемое имя
$app = @(
    @{ Id = "GitHub.GitHubDesktop";      Name = "GitHub Desktop" },
    @{ Id = "Microsoft.Edge";           Name = "Microsoft Edge" },
    @{ Id = "7zip.7zip";                 Name = "7-Zip" },
    @{ Id = "Microsoft.VisualStudioCode"; Name = "Visual Studio Code" }
)

foreach ($app in $apps) {
    Write-Host "--- Установка $($app.Name) ---"
    try {
        winget install --id $($app.Id) --silent `
            --accept-package-agreements --accept-source-agreements
        "[OK] $($app.Name) установлен" | Out-File $logFile -Append
    }
    catch {
        "[ОШИБКА] $($app.Name): $_" | Out-File $logFile -Append
    }
}

"=== Установка завершена: $(Get-Date) ===" | Out-File $logFile -Append
Write-Host "Готово. Лог: $logFile"
```

Главное отличие от bat-варианта список программ хранится не как отдельные вызовы подпрограммы, а как массив объектов, по которому проходит цикл `foreach`, а ошибки перехватываются конструкцией `try/catch`, а не проверкой кода возврата вручную. Функционально оба скрипта решают одну и ту же задачу; выбор конкретного инструмента зависит от того, какой из них привычнее использовать в конкретной организации.